Come impostare le richieste

Introduzione

Nelle lezioni precedenti abbiamo visto come definire **route statiche** e **dinamiche** in Flask e come gestire **richieste e risposte JSON**. Ora ci spostiamo su un concetto fondamentale per costruire **API RESTful**: i **verbi HTTP**.

Implementare i verbi HTTP in Flask

I verbi HTTP (`GET` , `POST` , `PUT` , `DELETE` , ecc.) definiscono **l'azione che il client vuole eseguire su una risorsa.**
In Flask, possiamo associare ogni route a uno o più verbi HTTP tramite il parametro `methods` .
In questo modo, il server può distinguere, ad esempio, una richiesta di lettura (`GET`) da una richiesta di creazione (`POST`).

Esempio di GET in Flask

Supponiamo di voler creare un endpoint per ottenere una lista di libri dal nostro server:

Python

```python
from flask import Flask, jsonify

app = Flask(__name__)

@app.route('/libri', methods=['GET'])
def get_libri():
    libri = [
        {'id': 1, 'titolo': '1984', 'autore': 'George Orwell'},
        {'id': 2, 'titolo': 'Il nome della rosa', 'autore': 'Umberto Eco'}
    ]
    return jsonify(libri), 200


```

Come funziona questo codice:

1. **Il client invia una richiesta HTTP** `GET` **all'endpoint** `/libri` **.**

2. **Flask intercetta la richiesta e chiama automaticamente la funzione `get_libri()`.**
3. **All'interno della funzione, viene costruita una lista di dizionari che rappresenta i dati dei libri.**
4. La funzione usa `jsonify()` **per convertire la lista in JSON, che è il formato standard per le risposte delle API REST.**
5. Flask restituisce la risposta al client insieme allo **status code 200**, che indica che la richiesta è andata a buon fine.

Gestire risposte semplici

Se la route restituisce una semplice stringa, Flask costruisce automaticamente una risposta HTTP:

Python

```python
@app.route('/')
def home():
    return "Hello World"
```

In questo caso, la stringa `"Hello World"` viene inviata al client con lo status code predefinito 200.
Questa introduzione ai verbi HTTP è fondamentale per comprendere come costruire API RESTful più complesse, in cui **diverse azioni su una stessa risorsa** (come leggere, creare, aggiornare o cancellare un libro) vengono gestite attraverso **route uniche e verbi HTTP appropriati**, migliorando chiarezza, manutenzione e compatibilità con strumenti web standard come la cache.

Effettuare richieste HTTP verso il server Flask

Dopo aver creato il server Flask con le route e le API, spesso è utile testare le API **da un client Python**. Per farlo possiamo usare la libreria `requests`, che:

- **permette di inviare richieste HTTP come `GET` o `POST` e ricevere le risposte.**
  Per installarla:

Bash

```bash
pip install requests
```

1. **Preparare le intestazioni (headers)**
   Le **intestazioni HTTP** servono a:

- **comunicare al server il tipo di dati che stiamo inviando e quello che ci aspettiamo in risposta.**
  Per le API REST in JSON, possiamo definire:

Python

```python
import requests
import json

headers = {
    'Content-Type': 'application/json',  # Dati inviati in formato JSON
    'Accept': 'application/json' # Ci aspettiamo la risposta in JSON
}
```

2. **Esempio di GET:**
   Supponiamo di voler leggere la lista di libri dal nostro server Flask:

Python

```python
response = requests.get(
    url="http://127.0.0.1:5000/libri",
    headers=headers
)
print("Risposta GET:", response.json()) #Stampa il json della risposta su terminale
```

Come funziona:

1. `requests.get()` i **nvia una richiesta HTTP GET all'endpoint** `/libri`**.**
2. **Flask intercetta la richiesta e restituisce i dati dei libri in formato JSON.**
3. `response.json()` **converte la risposta JSON in un oggetto Python (lista o dizionario) per poterla leggere o elaborare.**

In questo modo possiamo testare facilmente le nostre route di sola lettura.

3. **Esempio di POST**

- Se vogliamo **creare una nuova risorsa**, ad esempio un utente, utilizziamo il verbo `POST` e inviamo un payload JSON

Python

```python
payload = {
    "nome": "Mario",
    "cognome": "Rossi"
}

response_post = requests.post(
    url="http://127.0.0.1:5000/api/utenti",
    json=payload,
    headers=headers
)
print("Risposta POST:", response_post.json())
```

Come funziona:

1. **Il client invia un POST all'endpoint `/api/utenti` con i dati del nuovo utente.**
2. **Flask legge i dati con `request.get_json()` all'interno della route corrispondente.**
3. **Se la creazione della risorsa ha successo, il server restituisce i dati appena creati insieme allo status code `201 Created`, indicante che la risorsa è stata creata correttamente.**

4. Opzione avanzata con `json.dumps()`

In alternativa, possiamo serializzare manualmente il payload con `json.dumps()`. **Questo approccio offre maggiore controllo sul formato dei dati e sugli headers, utile per API più complesse**:

Python

```python
payload = {
    "nome": "Mario",
    "cognome": "Rossi"
}

response_post_dumps = requests.post(
    url="http://127.0.0.1:5000/api/utenti",
    data=json.dumps(payload),  # Serializza manualmente in JSON
```

```python
    headers=headers
)
print("Risposta POST:", response_post_dumps.json())
```

Questo metodo è spesso utilizzato quando si devono personalizzare gli headers o gestire payload complessi.

5. Esempio PUT

Il verbo `PUT` viene usato quando vogliamo **sovrascrivere completamente** una risorsa esistente.

Python

```python
from flask import Flask, request, jsonify

app = Flask(__name__)

libri = [
    {'id': 1, 'titolo': '1984', 'autore': 'George Orwell'},
    {'id': 2, 'titolo': 'Il nome della rosa', 'autore': 'Umberto Eco'}
]

@app.route('/libri/<int:libro_id>', methods=['PUT'])
def update_libro(libro_id):
    dati = request.get_json()
    libro = next((l for l in libri if l['id'] == libro_id), None)

    if not libro:
        return jsonify({'errore': f'Libro {libro_id} non trovato'}), 404

    # Aggiornamento completo
    libro['titolo'] = dati.get('titolo', libro['titolo'])
    libro['autore'] = dati.get('autore', libro['autore'])

    return jsonify(libro), 200

```

Logica:

1. **Il client invia una richiesta `PUT` a `/libri/<libro_id>` con tutti i dati aggiornati nel payload JSON.**

2. **Flask cerca il libro corrispondente nella “lista simile a database”.**
3. **Se il libro esiste, sovrascrive i campi con i nuovi valori.**
4. **Flask restituisce la risorsa aggiornata con status code 200.**

6. Metodo PATCH

`PATCH` serve quando vogliamo modificare **solo alcuni campi** di una risorsa, senza sovrascrivere l’intero oggetto.

Python

```python
@app.route('/libri/<int:libro_id>', methods=['PATCH'])
def patch_libro(libro_id):
    dati = request.get_json()
    libro = next((l for l in libri if l['id'] == libro_id), None)

    if not libro:
        return jsonify({'errore': f'Libro {libro_id} non trovato'}), 404

    # Aggiornamento parziale
    if 'titolo' in dati:
        libro['titolo'] = dati['titolo']
    if 'autore' in dati:
        libro['autore'] = dati['autore']

    return jsonify(libro), 200
```

Logica:

1. **Il client invia una richiesta `PATCH` con solo i campi da aggiornare.**
2. **Flask identifica il libro e aggiorna solo i campi forniti nel payload.**
3. **La risposta JSON contiene la risorsa aggiornata con status code 200.**

7. Metodo DELETE

Il verbo `DELETE` serve a **rimuovere una risorsa** dal server.

Python

```python
@app.route('/libri/<int:libro_id>', methods=['DELETE'])
def delete_libro(libro_id):
    global libri
    libro = next((l for l in libri if l['id'] == libro_id), None)

```

```python
    if not libro:
        return jsonify({'errore': f'Libro {libro_id} non trovato'}), 404

    # Rimuove il libro dalla lista
    libri = [l for l in libri if l['id'] != libro_id]

    return '', 204
```

Logica:

1. **Il client invia una richiesta `DELETE` all'endpoint `/libri/<libro_id>`.**
2. **Flask verifica se la risorsa esiste.**
3. **Se esiste, la elimina e restituisce status code 204 No Content, senza corpo della risposta.**
4. **Se non esiste, restituisce 404 Not Found.**

> **✏ Nota importante:**
>
> Prima di eseguire qualsiasi client Python per testare le API, **il server Flask deve essere attivo**, altrimenti le richieste falliranno. Prima si avvia il server (`python app.py`), poi si eseguono i client che inviano GET o POST.

Flusso completo di una richiesta HTTP

Quando un client interagisce con un server Flask tramite HTTP, la comunicazione segue un flusso ben definito:

1. **Creazione della richiesta da parte del client**
   - I **I client costruisce la richiesta HTTP, specificando il verbo (`GET`, `POST`, `PUT`, `DELETE`), eventuali headers e, se necessario, un payload JSON con i dati da inviare.**
2. **Arrivo della richiesta al server Flask**
   - **La richiesta viene ricevuta dal server Flask, che la intercetta e la instrada verso la funzione corrispondente alla route indicata nell'URL.**
3. **Esecuzione della logica della route**

- **La funzione associata alla route esegue le operazioni richieste: ad esempio, leggere dati da un database, filtrare risultati, creare o aggiornare una risorsa.**

4. **Preparazione della risposta**
   - **Flask converte il risultato della funzione in una risposta HTTP, generalmente in formato JSON, e assegna lo status code corretto (ad esempio `200 OK`, `201 Created` o `404 Not Found`).**
5. **Invio della risposta al client**
   - **Il server restituisce la risposta al client, che può elaborarla o mostrarla all'utente.**

> ⚠️ **Importante:** il server Flask deve essere **attivo e in esecuzione** prima che il client invii richieste, altrimenti la connessione fallirà.